

Metodi di Ottimizzazione per Big Data

DI MIRKO DAMIANO

Università degli studi Roma “Tor Vergata”

Corso di Laurea Magistrale *Ingegneria dell’Automazione*

Indice

1	Modelli e algoritmi di machine learning	2
1.1	Rischio atteso e rischio empirico	2
1.2	Minimi quadrati	6
1.3	Regressione logistica	10
1.3.1	Regressione logistica regolarizzata	13
1.3.2	Classificazione multiclasse	13
1.4	Support Vector Machines (SVM)	14
1.5	Reti neurali	16
1.5.1	Forward propagation	19
1.5.2	Back propagation	21
1.6	Metodi di ottimizzazione per il machine learning	23
1.7	Reti neurali per il deep learning generativo	25
1.8	Cenni sull’apprendimento non supervisionato	26

1 Modelli e algoritmi di machine learning

Il machine learning è una branca dell'intelligenza artificiale che si occupa di definire macchine artificiali capaci di apprendere le regole di funzionamento di un sistema, a partire dalle osservazioni.

Immaginiamo un sistema input-output:

- l'**input** (x) è l'ingresso fornito dal sistema;
- l'**output** (y) è l'uscita generata

e la legge $y = f(x)$ che lega tali elementi non è nota a priori; per questo motivo l'obiettivo del machine learning è costruire un **modello predittivo** $\varphi(x)$ capace di generare un output che approssimi il più possibile quello reale.

Osservazione 1. *C'è una differenza tra modello, la struttura matematica che dato l'input produce l'output, e algoritmo, il metodo iterativo di ottimizzazione*

I dati usati per far apprendere le regole al modello costituiscono il **training set** e avendo a che fare con modelli matematici, tali dati devono essere rappresentati opportunamente:

- i **campioni**: descritti come vettori di numeri reali in $\mathbb{R}^n$;
- le **features**: singole componenti x^i del vettore di input, per $i = 1, \dots, m$;
- le **labels**: rappresentano l'output y^i che si vuole stimare, per $i = 1, \dots, m$

Distinguiamo tra:

- **apprendimento supervisionato**: il training set è composto da coppie note di input-output (x^i, y^i) ;
- **apprendimento non supervisionato**: l'output dei campioni non è noto, per cui il modello deve trovare delle strutture intrinseche nei dati (*clustering*)

ed in particolare ci concentreremo sulla prima tipologia di classe, ove il training set viene espresso come segue:

$$\mathcal{T} = \{(x^1, y^1), \dots, (x^m, y^m)\}$$

In essa inoltre si hanno due principali classi di problemi:

1. **classificazione**: l'output y^i assume valori discreti, ossia rappresenta la classe di appartenenza di x^i ; nello specifico si ha:
 - classificazione binaria in cui si hanno solo 2 classi e le etichette y^i vengono definite come scalari, ad esempio in una diagnosi medica si usa $y^i = 1$ per rappresentare un tessuto sano e $y^i = 0$ per rappresentare un tessuto malato;
 - classificazione multiclasse in cui si hanno più di 2 classi e le etichette y^i vengono definite come vettori in cui tutte le componenti sono pari a 0, tranne una, corrispondente alla categoria effettiva, che assume valore 1 (*one-hot encoding*)
2. **regressione**: l'output y^i assume valori continui reali

1.1 Rischio atteso e rischio empirico

Costruire un modello di previsione significa scegliere una funzione di predizione $\varphi(x): \mathbb{R}^n \rightarrow \mathbb{R}^p$ tale che, per ogni coppia di input-output, il valore predetto sia sufficientemente simile al valore reale:

$$\varphi(x) \approx y$$

Ad esempio se consideriamo il dataset $\mathcal{T} = \{(1, 3), (4, 9), (-2, -3), (1.5, 4) \dots\}$ ove l'output dipende dall'input secondo la legge:

$$y = 2x + 1$$

una funzione del tipo $\varphi(x) = 1.9x + 1.05$ potrebbe fornire un buon modello.

Poichè i campioni quando entrano nel modello seguono una certa distribuzione di probabilità $P(x, y)$, vorremmo che la funzione φ minimizzi proprio la probabilità che si commetta un errore $P[\varphi(x) \neq y]$, rispetto a tutte le possibili realizzazioni input-output; ciò però è infattibile in pratica, in quanto richiede sia la minimizzazione rispetto ad un insieme di funzioni sia la conoscenza della probabilità $P(x, y)$.

Si definisca allora una famiglia di funzioni di predizione come segue:

$$\mathcal{H} = \{\varphi((\cdot), w): \mathbb{R}^n \rightarrow \mathbb{R}^p, w \in \mathbb{R}^v\}$$

parametrizzata dal vettore di pesi w , e a titolo di esempio supponiamo di voler risolvere un problema di classificazione binaria come quello di una diagnosi medica; una possibile soluzione per poter affermare se un campione x^i rappresenta un tessuto malato o meno è verificare se il punto (x_1, x_2) si trova sopra o sotto una determinata retta $f: w_1 x_1 + w_2 x_2 + b = 0$.

In base al segno assunto da essa si ha che, in forma compatta:

$$y = \text{sgn}(w^\top x + b)$$

per cui scegliendo diversi valori di w, b si ottengono diverse funzioni di predizione, tutte appartenenti alla famiglia di **classificatori lineari**:

$$\mathcal{H} = \{\text{sgn}(w^\top x + b), (w, b) \in \mathbb{R}^{n+1}\}$$

così da poter classificare i campioni correttamente.

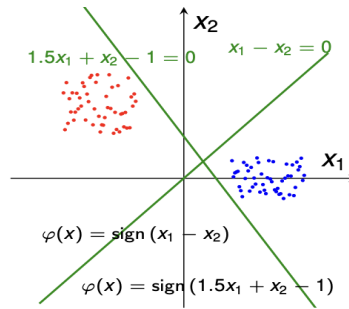


Figura 1.

Fissata allora tale famiglia $\mathcal{H}$, cerchiamo un valore corretto di (w, b) mediante una procedura di **training**, o addestramento, basata sulla minimizzazione di un'opportuna funzione di errore usando i campioni del training set. A tal proposito si definisca una **loss function** $\ell: \mathbb{R}^p \times \mathbb{R}^p \rightarrow [0, \infty)$ tale che $\ell(\varphi, y)$ misura lo scostamento tra il valore predetto e quello reale, ad esempio possiamo usare $\ell(\varphi, y) = \|\varphi - y\|^2$.

Inoltre per definizione:

- la loss function assume sempre valori non negativi ≥ 0 ;
- la loss function è nulla se e soltanto se $\varphi = y$, ossia il modello fa una predizione perfetta;
- più la predizione è sbagliata e più il valore della loss function diventa elevato

Definiamo allora il **rischio atteso** come il valore atteso dell'errore su tutte le possibili realizzazioni input-output (x, y) della distribuzione di probabilità $P(x, y)$, ossia misura quanto sbaglia in media un modello:

$$R_{\text{exp}}(w) = \mathbb{E}[\ell(\varphi(x, w), y)] = \int_{\mathbb{R}^n \times \mathbb{R}^p} \ell(\varphi(x, w), y) dP(x, y)$$

ove $\mathbb{R}^n$ è lo spazio degli input e $\mathbb{R}^p$ è lo spazio degli output.

Poichè la funzione di probabilità $P(x, y)$ non è nota, consideriamo l'errore medio che commettiamo esclusivamente sui campioni del training set, definendo il **rischio empirico** come:

$$R(w) = \frac{1}{m} \sum_{i=1}^m E_i(w) = \frac{1}{m} \sum_{i=1}^m \ell(\varphi(x^i, w), y^i)$$

ove $E_i(w)$ rappresenta l'errore commesso rispetto al campione i -esimo del training set.

Osservazione 2. Se il rischio empirico è alto il modello soffre di **underfitting**, mentre se si minimizza il rischio empirico si rischia di avere l'effetto opposto, noto come **overfitting**, cioè il modello fornisce ottimi risultati sui campioni del training set ma pessimi risultati sui nuovi dati.

Per cui l'obiettivo reale è la **generalizzazione**: il modello deve comportarsi bene su dati che non ha mai visto prima

Per legare matematicamente il rischio empirico (ciò che misuriamo) al rischio atteso (ciò che vorremmo minimizzare), si introduce la **VC Dimension** $d_{\mathcal{H}}$, che misura il massimo numero di punti che un modello può classificare correttamente in tutti i modi possibili; ovvero per ogni combinazione di etichette su un insieme di punti, esiste almeno una funzione (retta, curva) che li separa.

La teoria statistica dell'apprendimento stabilisce che, con probabilità $1 - \eta$, il rischio atteso è limitato superiormente da:

$$\text{rischio atteso} \leq \text{rischio empirico} + \text{VC confidence}$$

$$R_{\text{exp}}(w^*) \leq R(w^*) + \mathcal{O}\left(\sqrt{\frac{d_{\mathcal{H}}}{m} \log\left(\frac{m}{d_{\mathcal{H}}}\right) - \frac{\log \eta}{m}}\right)$$

ove $w^* \in \arg \min_w R(w)$ e $d_{\mathcal{H}} < m$. Si osserva pertanto che:

- se aumentiamo il numero di campioni m , la VC confidence diminuisce e il rischio empirico si avvicina a quello atteso;
- se aumentiamo la complessità del modello $d_{\mathcal{H}}$, il rischio empirico scende ma la VC confidence aumenta drasticamente, aprendo le porte all'overfitting

Esempio 3. Se consideriamo $m = 3$ punti nel piano non allineati, disposti come i vertici di un triangolo, esistono $2^3 = 8$ combinazioni possibili, ove ognuno di essi può essere classificato con una etichetta positiva o negativa.

Nel caso i campioni siano tutti positivi o tutti negativi, allora la retta può trovarsi al di sopra o al di sotto di essi, classificandoli in modo opportuno; se invece si ha un punto positivo e due negativi (o viceversa) si potrà sempre tracciare una retta che isola il punto singolo dagli altri due, essendo messi a triangolo.

Poichè tale retta riesce a separare tutte le 8 combinazioni possibili, diciamo che l'insieme dei classificatori lineari in $\mathbb{R}^n$, cioè $\mathcal{H} = \{\text{sgn}(w^\top x + b), (w, b) \in \mathbb{R}^{n+1}\}$, frantuma $n + 1$ punti; di conseguenza la VC dimension è almeno pari a $n + 1$.

Per dimostrare che essa si ferma a $n + 1$, dobbiamo verificare se una retta può frantumare 4 punti, i quali li disponiamo come i vertici di un quadrato; notiamo subito che se i vertici in alto a sinistra e in basso a destra sono positivi, mentre gli altri due sono negativi, non è possibile tracciare una retta che classifica i campioni in modo corretto

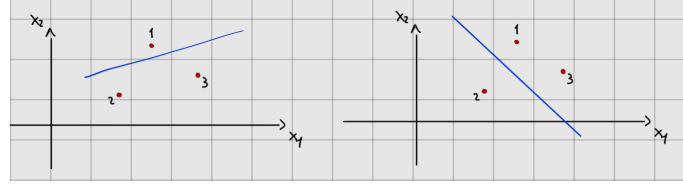


Figura 2.

Dalla precedente formula abbiamo dunque capito che il rischio empirico deve essere bilanciato con la complessità del modello per cui, invece di scegliere una generica famiglia di funzioni, possiamo considerare un certo numero di famiglie $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \dots \subseteq \mathcal{H}_q$ di complessità crescente, per poi scegliere la migliore di queste in termini di bilanciamento; ciò consiste nel minimizzare il **rischio strutturale**.

Consideriamo pertanto una famiglia di funzioni $\mathcal{H}$ e introduciamo una funzione di penalità $\Omega(w)$ che ne misura la complessità; il problema di addestramento si trasforma così nella minimizzazione del **rischio empirico regolarizzato**:

$$\min_w R(w) + \lambda \Omega(w)$$

ove $\lambda \geq 0$ è il parametro di regolarizzazione. Esso funge da bilancia, infatti un valore di λ vicino a zero dà poco peso alla complessità del modello rischiando overfitting, mentre un λ troppo elevato sposta l'attenzione sulla semplicità del modello piuttosto che sulla minimizzazione del rischio empirico.

Le due penalità principalmente utilizzate sono:

- $\|w\|_2^2$: la regolarizzazione ℓ_2 favorisce che i parametri siano piccoli in modulo, evitando che singole caratteristiche dominino eccessivamente;
- $\|w\|_1$: la regolarizzazione ℓ_1 promuove la sparsità, spingendo molte componenti del vettore dei pesi w a diventare zero. Nei modelli lineari agisce come una *feature selection*, eliminando le caratteristiche irrilevanti, diminuendo la complessità del modello ma aumentando il rischio empirico

Per decidere invece quale valore assegnare al parametro λ , definiamo una strategia di **cross validation**:

1. il dataset viene diviso in tre blocchi separati: *training set*, per calcolare i pesi w , *validation set*, per valutare le performance dei vari λ e *test set* per stimare l'errore reale finale;
2. si testa una griglia di valori $\lambda_1, \dots, \lambda_q$ e per ciascuno di essi si addestra il modello sul training set:

$$w \in \arg \min \{R(w) + \lambda_i \Omega(w)\}$$

e si misura l'errore ottenuto sul validation set;

3. si sceglie λ come il parametro con l'errore più piccolo e si usa il test set per stimare l'errore del modello

Una variante più sofisticata **k-fold validation**, invece, prevede la divisione del training set in K blocchi, ove a rotazione un blocco funge da validation set e i restanti $K - 1$ da training set; l'errore finale per quel valore di λ sarà la media dei K tentativi.

1.2 Minimi quadrati

Consideriamo un problema di regressione lineare, in cui vogliamo predire un'uscita continua $y \in \mathbb{R}$, del tipo:

$$y(x) = w^\top x + b + \epsilon$$

a partire da un vettore di caratteristiche $x \in \mathbb{R}^n$, con ϵ rumore indipendente dai dati. Di conseguenza la funzione di predizione avrà una forma lineare:

$$\varphi(x, (w, b)) = w^\top x + b$$

ove ricordiamo che $w \in \mathbb{R}^n$ è il vettore dei pesi e $b \in \mathbb{R}$ è l'intercetta.

Graficamente si ha una retta data dalla funzione φ (in rosso) e i campioni (in blu):

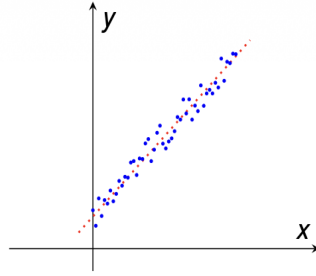


Figura 3.

Si vuole dunque calcolare (w^*, b^*) tali per cui la funzione di predizione approssima meglio l'andamento dell'output $y(x)$; per fare ciò definiamo una loss function **quadratica** del tipo:

$$\ell(\varphi, y) = (\varphi - y)^2$$

per misurare l'errore commesso su un singolo campione, così che il rischio empirico è dato dalla funzione di **minimi quadrati**:

$$R(w) = \frac{1}{m} \sum_{i=1}^m \ell(\varphi(x^i, w), y^i) = \frac{1}{m} \sum_{i=1}^m (y^i - (w^\top x^i + b))^2$$

Mettendo insieme la loss quadratica calcolata su tutto il training set e aggiungendo un termine di regolarizzazione ℓ_2 , si ottiene un modello di **minimi quadrati regolarizzato**:

$$\min_{w \in \mathbb{R}^n, b \in \mathbb{R}} \left\{ f(w, b) = \frac{1}{m} \sum_{i=1}^m (y^i - (w^\top x^i + b))^2 + \lambda \|w\|^2 \right\}$$

Per capire come risolvere tale problema, facciamo notare anzitutto che è possibile semplificarlo eliminando la variabile b dal problema di ottimizzazione; supponiamo di aver trovato la soluzione ottima (w^*, b^*) , la quale è tale che:

$$\begin{aligned} \frac{\partial f(w^*, b^*)}{\partial b} &= \frac{2}{m} \sum_{i=1}^m ((w^*)^\top x^i + b^* - y^i) \\ &\stackrel{!}{=} 0 \\ &\Downarrow \\ b^* &= \frac{1}{m} \sum_{i=1}^m y^i - \frac{1}{m} \sum_{i=1}^m (w^*)^\top x^i \\ &= \frac{1}{m} \sum_{i=1}^m y^i - \sum_{j=1}^n w_j^* \left(\frac{1}{m} \sum_{i=1}^m x_j^i \right) \end{aligned}$$

ottenendo dunque che $b^* = 0$ se i dati sono **centrati**, cioè se:

$$\frac{1}{m} \sum_{i=1}^m y^i = 0, \quad \frac{1}{m} \sum_{i=1}^m x_j^i = 0, \quad j = 1, \dots, n$$

per cui possiamo omettere l'intercetta dal modello, sottraendo preliminarmente le medie.

Osservazione 4. Supponiamo di aver osservato le uscite y^i , la cui media è data da:

$$\bar{y} = \frac{1}{m} \sum_{i=1}^m y^i$$

e definiamo dei nuovi dati osservati fittizi, $\hat{y}^i \triangleq y^i - \bar{y}$, ma strutturalmente centrati; infatti si ha che:

$$\frac{1}{m} \sum_{i=1}^m \hat{y}^i = \frac{1}{m} \sum_{i=1}^m y^i - \bar{y} = \bar{y} - \bar{y} = 0$$

mostrando così che per centrare i dati è necessario sottrarli alla loro media

Definendo ora la matrice $A = (x^1 \dots x^m)^\top \in \mathbb{R}^{m \times n}$, ove ogni riga corrisponde a un campione di input, si ha un nuovo problema da minimizzare:

$$\min_{w \in \mathbb{R}^n} \left\{ f(w) = \frac{1}{m} \sum_{i=1}^m (y^i - w^\top x^i)^2 + \lambda \|w\|^2 = \frac{1}{m} \|Aw - y\|^2 + \lambda \|w\|^2 \right\}$$

ove il vettore Aw restituisce un vettore contenente tutte le predizioni del modello e y vettore delle uscite reali.

Si fa inoltre osservare come esso sia un problema quadratico, in quanto:

$$\begin{aligned} f(w) &= \frac{1}{m} \|Aw - y\|^2 + \lambda \|w\|^2 \\ &= \frac{1}{m} (Aw - y)^\top (Aw - y) + \lambda w^\top w \\ &= \frac{1}{m} w^\top A^\top Aw - \frac{2}{m} (A^\top y)^\top w + \frac{1}{m} \|y\|^2 + \lambda w^\top w \\ &= w^\top \left(\frac{1}{m} A^\top A + \lambda I \right) w - \frac{2}{m} (A^\top y)^\top w + \frac{1}{m} \|y\|^2 \end{aligned}$$

per cui possiamo semplicemente calcolare gradiente e matrice Hessiana:

$$\begin{aligned} \nabla f(w) &= \left(\frac{1}{m} A^\top A + \lambda I \right) w - \frac{2}{m} (A^\top y)^\top = \frac{2}{m} A^\top (Aw - y) + 2\lambda w \\ \nabla^2 f(w) &= \frac{2}{m} A^\top A + 2\lambda I \end{aligned}$$

Quindi la funzione è **convessa**, ma se la matrice A non ha rango massimo, ad esempio se si hanno più caratteristiche che dati, $n > m$, la soluzione potrebbe non essere unica; se invece $\lambda > 0$, la matrice Hessiana diventa strettamente definita positiva e la funzione **fortemente convessa**, garantendo così l'esistenza di un'unica soluzione ottima globale w^* .

Essendo il problema differenziabile, convesso e non vincolato, il punto di minimo globale si trova imponendo la condizione di stazionarietà, ovvero annullando il gradiente:

$$\begin{aligned}
\nabla f(w) &= \frac{2}{m} A^\top (A w - y) + 2 \lambda w \\
&= \frac{2}{m} A^\top A - \frac{2}{m} A^\top y + 2 \lambda w \\
&\stackrel{!}{=} 0 \\
&\Downarrow \\
\frac{2}{m} A^\top y &= \frac{2}{m} A^\top A + 2 \lambda w \\
\left(\frac{2}{m} A^\top y \right) \frac{m}{2} &= \left(\frac{2}{m} A^\top A + 2 \lambda w \right) \frac{m}{2} \\
A^\top y &= (A^\top A + m \lambda I) w \\
&\Downarrow \\
w^\star &= (A^\top A + m \lambda I)^{-1} A^\top y
\end{aligned}$$

Pertanto trovare una soluzione in forma chiusa significa che non abbiamo bisogno di un algoritmo iterativo per trovare l'ottimo, ma basta applicare tale formula matematica direttamente sui dati.

Osservazione 5. *Con dataset molto grandi, la memorizzazione di $A^\top A$ non è fattibile e occorre usare metodi adatti, ad esempio Gauss-Seidel*

Un'alternativa alla regolarizzazione ℓ_2 , si basa sulla norma ℓ_1 ottenendo un problema **LASSO**:

$$\min_{w \in \mathbb{R}^n} \left\{ f(w) = \frac{1}{m} \|A w - y\|^2 + \lambda \|w\|_1 \right\}$$

inducendo così sparsità nella soluzione finale $w^\star$, cioè essa avrà molte componenti nulle. In tal caso però la funzione obiettivo è convessa ma non strettamente, in quanto la norma ℓ_1 è una funzione convessa, ma lineare a tratti.

Di conseguenza, l'intero problema LASSO perde la proprietà di stretta convessità (a meno che la matrice A non abbia proprietà particolari); questo significa che la soluzione ottima potrebbe non essere unica. Inoltre la funzione valore assoluto $|w_i|$ presenta un punto angoloso in corrispondenza dello zero, ove la funzione non è differenziabile e il gradiente non esiste.

Per superare allora l'ostacolo della non differenziabilità causata dal valore assoluto, possiamo esprimere ogni singola componente w_i del vettore dei pesi come la differenza di due quantità strettamente non negative:

$$w_i = w_i^+ - w_i^-, \quad w_i^+, w_i^- \geq 0$$

ove $w_i^+ = \max\{w, 0\}$, $w_i^- = \max\{-w, 0\}$, così che il valore assoluto può scriversi linearmente come:

$$|w_i| = w_i^+ + w_i^-$$

Sostituendo si ottiene una funzione obiettivo completamente differenziabile:

$$\begin{cases} \min_{w^+, w^- \in \mathbb{R}^n} \left\{ \frac{1}{m} \|A(w^+ - w^-) - y\|^2 + \lambda \sum_{i=1}^n (w^+ - w^-)_i \right\} \\ w^+, w^- \geq 0 \end{cases}$$

in cui il numero di variabili è raddoppiato da n a $2n$.

Una formulazione LASSO viene anche utilizzata per risolvere sistemi lineari **sottodeterminati** in cui il numero di feature è molto maggiore del numero di campioni, $m \ll n$:

$$A w = y$$

ove $A \in \mathbb{R}^{m \times n}$, $y \in \mathbb{R}^m$; poichè in tale contesto la matrice A è non quadrata allora si avranno $\infty^{n-\text{rank}(A)}$ soluzioni, tra le quali si cercano quelle che soddisfano il vincolo di sparsità, ovvero:

$$\begin{cases} \min_{w \in \mathbb{R}^n} \|Aw - y\|^2 \\ \|w\|_1 < \varepsilon \end{cases}$$

Equivalentemente si può aggiungere un termine di regolarizzazione ℓ_1 così che si possa utilizzare il seguente modello:

$$\min_{w \in \mathbb{R}^n} \|Aw - y\|^2 + \lambda \|w\|_1$$

quando si cerca un vettore w sparso tale che $Aw \approx y$.

Esempio 6. Consideriamo un segnale acustico:

$$z(t) = \sin(1394 \pi t) + \sin(3266 \pi t)$$

della durata di 0.125 secondi, campionato ad una frequenza di 40000 Hz , per cui il numero di campioni è dato da $n = 40000 \times 0.125 = 5000$.

Poichè tale segnale non è sparso, applichiamo la **trasformata discreta del coseno** (DCT):

$$\sum_{j=0}^{n-1} c_{ij} w(j) = z(i), \quad i = 0, \dots, n-1$$

o in forma compatta:

$$Cw = z$$

ove la matrice C contiene tutti i coefficienti c_{ij} ; in particolare la DCT è una funzione invertibile per cui si può facilmente ottenere il segnale trasformato:

$$w = C^{-1} z$$

Si ottiene dunque una rappresentazione sparsa del segnale originale:

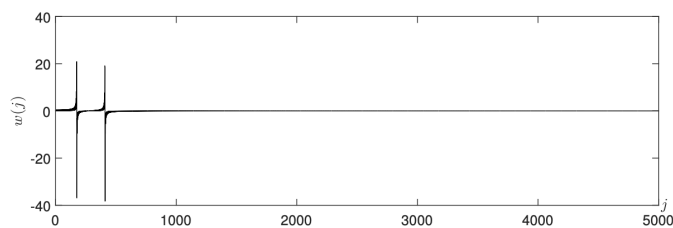


Figura 4.

mostrando che esso si rivela composto da pochissimi coefficienti non nulli.

Sotto-campionando il segnale in modo casuale, definiamo $y \in \mathbb{R}^n$ scegliendo $m \ll n$ di elementi e $A \in \mathbb{R}^{m \times n}$ come la sottomatrice di C ottenuta mantenendo solo le righe che corrispondono agli elementi del vettore y . Allora ricostruire il segnale significa trovare $w \in \mathbb{R}^n$ tale che:

$$Aw \approx y$$

per cui il LASSO riesce a ricostruire perfettamente il segnale originale sia nel dominio delle frequenze che del tempo.

Mostriamo i risultati con $m = 500$ campionamenti:

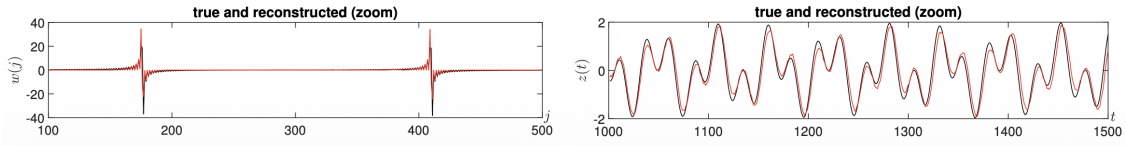


Figura 5.

È possibile estendere il modello a problemi multiclasse mediante una strategia **One-vs-All**: consideriamo P classi $\{c_1, \dots, c_p\}$ e un insieme di dati $\{(x^1, y^1), \dots, (x^m, y^m)\}$, ove x^j è un vettore di feature e y^j è la classe a cui x^j appartiene.

Per ogni classe i , si costruisce un classificatore binario in cui le etichette valgono 1 se il campione appartiene alla classe i , e -1 se appartiene a qualunque altra classe:

$$\hat{y}^j \begin{cases} 1, & \text{se } x^j \in c_i \\ -1, & \text{altrimenti} \end{cases}$$

Si ottengono così p modelli distinti $(w^1, b^1), \dots, (w^p, b^p)$ ove un nuovo campione x viene assegnato alla classe che massimizza il valore dell'output lineare:

$$i = \text{Arg} \max_{j=1, \dots, p} \{(w^j)^\top x + b^j\}$$

1.3 Regressione logistica

La regressione logistica è un modello di classificazione **binaria**, $y_i \in \{-1, 1\}$, e assumiamo che y segua una **distribuzione di Bernoulli**, ossia dato un campione x^j in ingresso si ha che:

$$\begin{cases} P(y=1|x) = p(x) \\ P(y=-1|x) = 1 - p(x) \end{cases}$$

ove $p(x)$ è una probabilità che dipende da x attraverso la **funzione logistica**:

$$p(x) = \frac{e^{w^\top x + b}}{1 + e^{w^\top x + b}} = \frac{1}{1 + e^{-(w^\top x + b)}}, \quad w \in \mathbb{R}^n, b \in \mathbb{R}$$

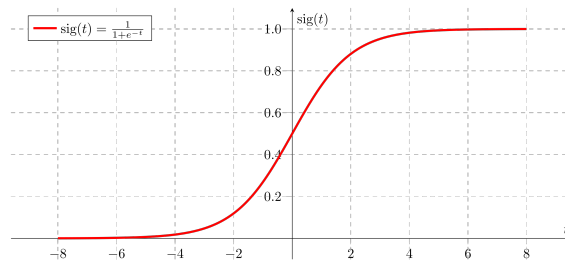


Figura 6.

Essa prende un valore arbitrario $z = w^\top x + b \in \mathbb{R}$ e lo mappa schiacciandolo nell'intervallo $(0, 1)$, così da consentire di interpretare l'output direttamente come una stima della probabilità che il punto appartenga alla classe $y = 1$.

Osservazione 7. Osserviamo che nel caso dei minimi quadrati non si poteva parlare di probabilità ma solo di un valore di predizione continuo; invece la regressione logistica fornisce una misura quantitativa di incertezza.

Tuttavia nella classica regressione lineare si tenta di modellare direttamente la risposta usando una combinazione lineare delle caratteristiche:

$$\text{output} = w^\top x + b$$

ma il termine $w^\top x + b$ può assumere un qualsiasi valore reale nell'intervallo $(-\infty, +\infty)$, mentre una probabilità $p(x)$ deve stare nell'intervallo $[0, 1]$.

Per questo motivo si introducono gli **ODD** e il **Logit**: l'ODD, o rapporto di probabilità, è definito come segue:

$$\text{ODD} \triangleq \frac{p(x)}{1-p(x)} = \frac{\text{probabilità che un evento accada}}{\text{probabilità che un evento non accada}} = e^{w^\top x + b} \in [0, +\infty)$$

e applicando il logaritmo naturale ad entrambi i membri si ha che il termine logit è lineare in x :

$$\text{logit} \triangleq \log(\text{ODD}) = \log\left(\frac{p(x)}{1-p(x)}\right) = w^\top x + b$$

così da trasformare un problema di stima probabilistica, vincolato tra 0 ed 1, in un problema di regressione lineare non vincolato su tutto $\mathbb{R}$.

Se assumiamo che i campioni nel training set $\mathcal{T}$ siano indipendenti e identicamente distribuiti, allora la probabilità di osservare la corretta etichetta y^i è data dalla **funzione di verosimiglianza**:

$$L((w, b), \mathcal{T}) = P(y^1|x^1) P(y^2|x^2) \dots P(y^m|x^m) = \prod_{i=1}^m P(y^i|x^i)$$

Per poter trovare dunque i parametri ottimi (w, b) in modo da massimizzare la probabilità delle osservazioni in $\mathcal{T}$, cioè minimizzare l'errore sul training set, usiamo le proprietà dei logaritmi per scrivere che:

$$\begin{aligned} (w^*, b^*) &\in \text{Argmax}_{(w, b) \in \mathbb{R}^{n+1}} \prod_{i=1}^m P(y^i|x^i) \\ &= \text{Argmax}_{(w, b) \in \mathbb{R}^{n+1}} \log\left(\prod_{i=1}^m P(y^i|x^i)\right) \\ &= \text{Argmax}_{(w, b) \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log(P(y^i|x^i)) \\ &= \text{Argmin}_{(w, b) \in \mathbb{R}^{n+1}} \left[-\sum_{i=1}^m \log(P(y^i|x^i)) \right] \end{aligned}$$

così che il problema da risolvere diventa quello di minimizzare la funzione di log-verosimiglianza cambiata di segno, in quanto massimizzare un prodotto è computazionalmente complesso:

$$\min_{(w, b) \in \mathbb{R}^{n+1}} \left[-\sum_{i=1}^m \log(P(y^i|x^i)) \right]$$

Cerchiamo ora di ottenere una forma più compatta della precedente relazione, notando che:

$$P(y^i|x^i) = \begin{cases} p(x^i) = \frac{1}{1 + e^{-(w^\top x^i + b)}}, & \text{se } y^i = 1 \\ 1 - p(x^i) = \frac{1}{1 + e^{w^\top x^i + b}}, & \text{se } y^i = -1 \end{cases}$$

ossia:

$$P(y^i|x^i) = \frac{1}{1 + e^{-y^i(w^\top x^i + b)}}, \quad \forall (x^i, y^i) \in \mathcal{T}$$

per cui usando ancora le proprietà dei logaritmi, il problema da risolvere è:

$$\min_{(w,b) \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log(1 + e^{-y^i(w^\top x^i + b)})$$

In particolare la funzione obiettivo che si ottiene è **strettamente convessa**, infatti:

$$\begin{aligned} - \quad \nabla(-\log(L((w,b), \mathcal{T}))) &= \sum_{i=1}^m y^i \left(\frac{1}{1 + e^{-y^i(w^\top x^i + b)}} - 1 \right) \begin{pmatrix} x^i \\ 1 \end{pmatrix}; \\ - \quad \nabla^2(-\log(L((w,b), \mathcal{T}))) &= \begin{pmatrix} (x^1)^\top & 1 \\ \vdots & \vdots \\ (x^m)^\top & 1 \end{pmatrix}^\top D \begin{pmatrix} (x^1)^\top & 1 \\ \vdots & \vdots \\ (x^m)^\top & 1 \end{pmatrix} = A^\top D A \end{aligned}$$

ove $A \in \mathbb{R}^{m \times (n+1)}$ e $D \in \mathbb{R}^{m \times m}$ è una matrice diagonale definita come segue:

$$D = \begin{pmatrix} \frac{1}{1 + e^{-y^1(w^\top x^1 + b)}} \left(1 - \frac{1}{1 + e^{-y^1(w^\top x^1 + b)}} \right) & 0 & \cdots & 0 \\ 0 & \ddots & \cdots & 0 \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & \frac{1}{1 + e^{-y^m(w^\top x^m + b)}} \left(1 - \frac{1}{1 + e^{-y^m(w^\top x^m + b)}} \right) \end{pmatrix}$$

e ciò garantisce che ogni punto stazionario sia un minimo globale **unico**.

Facciamo ora un confronto tra i minimi quadrati e la regressione logistica usando la classificazione binaria; consideriamo lo scenario in cui si abbiano un certo set di campioni x^i (in verde), ciascuno dei quali viene etichettato come classe 0/1, ed ove per classificare un nuovo punto si definisce un valore di soglia pari a 0.5:

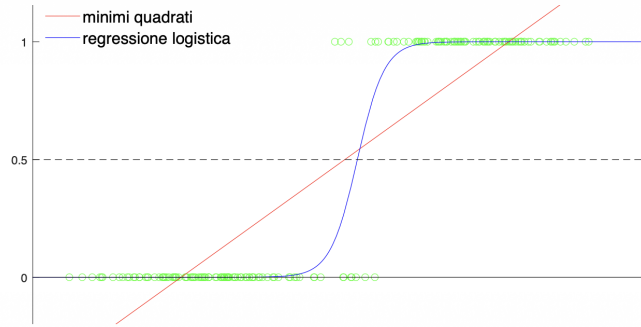


Figura 7.

- **MINIMI QUADRATI:** sappiamo che la retta rossa rappresenta il modello di regressione lineare

$$h(x, (w, b)) = w^\top x + b$$

e proiettando il punto di incrocio sull'asse x si ottiene il valore di soglia dell'input per i minimi quadrati, per cui a sinistra di tale valore la retta si trova sotto l'altezza 0.5 e il modello assegna punti alla classe 0; invece a destra di tale valore la retta si trova sopra l'altezza 0.5 e il modello assegna punti alla classe 1;

- **REGRESSIONE LOGISTICA:** ragionando allo stesso modo per la curva blu, a sinistra del valore x di soglia la probabilità che il campione appartenga alla classe 1 è inferiore al 50% per cui si assegna il dato alla classe 0, e viceversa

In più si osserva come i due punti di incrocio con la linea $y = 0.5$ non coincidono, affermando che la soglia di decisione cambia in funzione del modello scelto; la differenza sostanziale sta però nel fatto che la regressione logistica restituisce la probabilità che un campione appartenga ad una certa classe misurando quanto il modello sia sicuro della propria scelta, mentre per i minimi quadrati ciò non vale perchè se la retta rossa per un certo campione x restituisce $y = 1.4$ non ha senso dire che c'è il 140% di probabilità.

Se i dati sono **linearmente separabili**, ossia esiste un iperpiano capace di separare perfettamente i punti della prima classe da quelli della seconda classe, analizziamo cosa accade alla funzione di costo:

$$\min_{(w,b) \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log(1 + e^{-y^i(w^\top x^i + b)})$$

1. la funzione $\log(1 + e^{-y^i(w^\top x^i + b)})$ tende a zero quando il suo argomento $e^{-y^i(w^\top x^i + b)}$ tende a $+\infty$;
2. se si moltiplica l'intero vettore dei pesi w^* e il bias b^* per un valore arbitrariamente grande $c > 0$, l'argomento diventa:

$$y^i(c w^\top x^i + c b) = c y^i(w^\top x^i + b)$$

3. man mano che si fa crescere $c \rightarrow +\infty$ si ha che $\log(1) = 0$ e la funzione di costo complessiva tende a zero

Tuttavia per raggiungere esattamente il valore zero la norma del vettore dei pesi $\|w\|$ e il bias b dovrebbero tendere a $+\infty$, per cui non esiste un punto finito (w^*, b^*) che sia un minimo per la funzione.

1.3.1 Regressione logistica regolarizzata

Per risolvere questo problema e rendere la soluzione finita ed unica, si aggiunge un termine di regolarizzazione ℓ_2 alla funzione obiettivo:

$$\min_{(w,b) \in \mathbb{R}^{n+1}} \sum_{i=1}^m \log(1 + e^{-y^i(w^\top x^i + b)}) + \lambda \|w\|_2^2, \quad \lambda > 0$$

in modo tale che non si può più spingere $\|w\| \rightarrow +\infty$ per azzerare la prima parte della perdita, perchè questo farebbe esplodere il termine aggiuntivo; inoltre la funzione diventa strettamente convessa e coerciva, garantendo l'esistenza di un unico valore ottimo finito (w^*, b^*) .

1.3.2 Classificazione multiclasse

Per la classificazione multiclasse una prima possibilità è usare un approccio *one-vs-all*: si addestrano P classificatori binari $(w^1, b^1), \dots, (w^p, b^p)$ e ogni campione x viene assegnato alla classe di appartenenza con probabilità maggiore:

$$i \in \text{Argmax}_{j=1, \dots, p} P(y^j | x)$$

ove per il calcolo della probabilità $P(y^j | x)$ si applica la funzione **soft-max**:

$$P(y^j | x) = \frac{e^{((w^j)^\top x + b^j)}}{\sum_{h=1}^p e^{((w^h)^\top x + b^h)}}, \quad j = 1, \dots, p$$

Una seconda possibilità consiste invece nel supporre di utilizzare sempre la funzione soft-max ma in tal caso le etichette y^i diventano vettori dove solo la componente della classe vera vale 1 e le altre 0, ossia si pone $y_j^i = 1$ se il campione x^i appartiene alla classe j .

Allora nella funzione da minimizzare, presentata precedentemente, si ha che:

$$\log(P(y^i|x^i)) = \sum_{j=1}^p y_j^i \log(P(y_j^i|x^i))$$

così da considerare la probabilità moltiplicata solo quando $y_j^i = 1$; sostituendo pertanto tale relazione nel problema di minimizzazione si ottiene una funzione **cross-entropy**:

$$\min_{(w^1, b^1), \dots, (w^p, b^p) \in \mathbb{R}^{n+1}} \left[- \sum_{i=1}^m \sum_{j=1}^p y_j^i \log(P(y_j^i|x^i)) \right]$$

1.4 Support Vector Machines (SVM)

Consideriamo un problema di classificazione binaria linearmente separabile in cui le etichette assumono convenzionalmente i valori $y_i \in \{-1, 1\}$; in tal caso le SVM cercano il miglior iperpiano possibile chiedendo non solo che divida correttamente i punti, ma che mantenga anche una distanza di sicurezza, o **soglia** $\varepsilon > 0$, rispetto ai dati delle due classi.

Si vuole allora costruire un classificatore lineare tale che, per ogni $i = 1, \dots, m$, si abbia:

$$\begin{cases} w^\top x^i + b \geq \varepsilon, & \text{se } y^i = 1 \\ w^\top x^i + b \leq -\varepsilon, & \text{se } y^i = -1 \end{cases}$$

e senza perdita di generalità si pone $\varepsilon = 1$, ottenendo un'unica forma compatta:

$$y^i(w^\top x^i + b) \geq 1, \quad i = 1, \dots, m$$

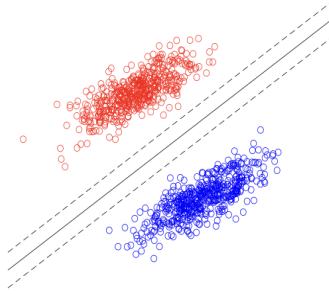


Figura 8.

Osservazione 8. Dato un iperpiano $w^\top x + b = 0$, la distanza ortogonale di un generico punto x^i dall'iperpiano stesso è data dal rapporto:

$$\frac{|w^\top x^i + b|}{\|w\|}$$

Dato ora un training set $\mathcal{T}$ e un iperpiano $w^\top x + b = 0$, definiamo il **margin** ρ come la minima distanza tra l'iperpiano e tutti i punti di $\mathcal{T}$:

$$\rho = \min_{i=1, \dots, m} \frac{|w^\top x^i + b|}{\|w\|}$$

Lemma 9. *Se tutti i punti sono contenuti in una sfera di raggio D , la VC dimension $d_{\mathcal{H}}$ dei classificatori lineari con margine ρ è limitata superiormente da:*

$$d_{\mathcal{H}} \leq \min \left\{ \left\lceil \frac{D^2}{\rho^2} \right\rceil, n \right\} + 1$$

Più il margine ρ è grande, più la VC dimension $d_{\mathcal{H}}$ è piccola e minore sarà il bound sul rischio atteso

L'idea alla base delle SVM è quella di calcolare l'iperpiano di separazione in modo da **massimizzare il margine**, per cui il problema da risolvere è descritto dalla seguente struttura:

$$\begin{cases} \max_{(w,b) \in \mathbb{R}^{n+1}} \left\{ \min_{i=1, \dots, m} \frac{|w^\top x^i + b|}{\|w\|} \right\} \\ y^i(w^\top x^i + b) \geq 1, \quad i = 1, \dots, m \end{cases}$$

In particolare per i punti che si trovano sul bordo del margine si ha che $\rho = \frac{1}{\|w\|}$ e massimizzare tale margine equivale a minimizzare la norma $\|w\|$; si può pertanto mostrare che la soluzione ottima del precedente problema è unica ed è anche l'unica soluzione ottima del problema **primale**:

$$\begin{cases} \min_{(w,b) \in \mathbb{R}^{n+1}} \left\{ \frac{1}{2} \|w\|^2 \right\} \\ y^i(w^\top x^i + b) \geq 1, \quad i = 1, \dots, m \end{cases}$$

Osservazione 10. *Si tratta di un problema di Programmazione Quadratica convesso con vincoli di disuguaglianza lineari*

In pratica però è spesso preferibile risolvere il problema **duale**, per cui sfruttando la teoria della dualità di Lagrange, si introduce un moltiplicatore $\alpha_i \geq 0$ per ciascun vincolo:

$$\begin{cases} \min_{\alpha \in \mathbb{R}^m} \left\{ \frac{1}{2} \alpha^\top Q \alpha - \sum_{i=1}^m \alpha_i \right\} \\ \sum_{i=1}^m y^i \alpha_i = 0 \\ \alpha_i \geq 0, \quad i = 1, \dots, m \end{cases}$$

ove $Q \in \mathbb{R}^{m \times m}$ è una matrice tale che $Q_{i,j} = (x^i)^\top x^j$ per $i, j = 1, \dots, m$.

Osservazione 11. *Tale problema si può risolvere usando un metodo a blocchi a 2 variabili, passando per Gauss-Southwell per scegliere i blocchi*

Si può dunque mostrare che, una volta calcolata la soluzione ottima α^* , possiamo ricostruire la soluzione (w^*, b^*) del problema primale, con:

$$w^* = \sum_{i=1}^m \alpha_i^* y^i x^i$$

e l'intercetta b^* si ricava imponendo l'uguaglianza $y^i(w^{*\top} x^i + b^*) = 1$ per qualunque vettore di supporto.

In particolare i punti del training set con $\alpha_i^* > 0$ sono detti **vettori di supporto** e sono gli unici punti che toccano il margine $y^i(w^\top x^i + b) = 1$ e determinano l'iperpiano finale.

Se i dati non sono linearmente separabili non esiste un iperpiano che possa separare i dati senza commettere errori; allora per ogni campione x^i si ammette un errore quantificato da una variabile di slack $\xi_i \geq 0$ e l'errore sul singolo campione viene misurato tramite la **hinge loss function**:

$$\max \{0, 1 - y^i(w^\top x^i + b)\}, \quad i = 1, \dots, m$$

Infatti se il punto è ben classificato e oltre il margine, la perdita è zero, altrimenti la perdita cresce linearmente con la distanza dal margine.

Di conseguenza il primale assume la seguente forma di problema di minimizzazione del rischio empirico regolarizzato:

$$\min_{(w,b) \in \mathbb{R}^{n+1}} \left\{ \frac{1}{m} \sum_{i=1}^m \max \{0, 1 - y^i(w^\top x^i + b)\} + \lambda \|w\|^2 \right\}$$

Esso è un problema convesso ma **non differenziabile**; possiamo però riformularlo come segue:

$$\begin{cases} \min_{(w,b) \in \mathbb{R}^{n+1+m}} \frac{1}{m} \sum_{i=1}^m \xi_i + \lambda \|w\|^2 \\ y^i(w^\top x^i + b) \geq 1 - \xi_i, & i = 1, \dots, m \\ \xi_i \geq 0, & i = 1, \dots, m \end{cases}$$

e anche in tal caso risolvendo il duale, l'unica modifica alla funzione obiettivo duale consiste nell'aggiunta di un upper bound sulle variabili duali α_i :

$$\begin{cases} \min_{\alpha \in \mathbb{R}^m} \left\{ \frac{1}{2} \alpha^\top Q \alpha - \sum_{i=1}^m \alpha_i \right\} \\ \sum_{i=1}^m y^i \alpha_i = 0 \\ 0 \leq \alpha_i \leq C, & i = 1, \dots, m \end{cases}$$

ove $C = \frac{1}{2m\lambda}$ è un parametro di bilanciamento.

Osservazione 12. *Nel caso di classificazione non lineare, quando la frontiera di separazione non è un iperpiano lineare, si mappa lo spazio originario degli input $x \in \mathbb{R}^n$ in uno spazio di caratteristiche a dimensione più alta mediante una funzione non lineare $\phi(x)$.*

*Solitamente nel problema duale i vettori di input compaiono sotto forma di prodotti scalari $(x^i)^\top x^j$ e invece di calcolare esplicitamente la trasformazione ad alta dimensione $\phi(x^i)$, si sostituisce il prodotto scalare $\phi(x^i)^\top \phi(x^j)$ con una **funzione kernel** $K(x^i, x^j)$ direttamente nello spazio di partenza:*

$$K(x^i, x^j) = \phi(x^i)^\top \phi(x^j)$$

In tal modo il classificatore trova una superficie di separazione non lineare nello spazio originario dei dati, rimanendo un iperpiano lineare nello spazio trasformato $\phi(x)$

1.5 Reti neurali

Le reti neurali sono un modello in cui le entità elementari, i **neuroni**, ricevono l'informazione, la elaborano e la trasmettono alle altre unità; il **Perceptron** è il modello matematico ispirato al funzionamento del neurone biologico ed è costituito da un solo neurone con una sola uscita.

Nello specifico il neurone riceve in ingresso un vettore di caratteristiche $(x_1 \dots x_n)^\top$ e a ciascuno di essi associa un peso w_i per poi calcolare la combinazione lineare degli ingressi, più un termine noto w_0 detto **bias**:

$$z = \sum_{i=1}^n w_i x_i + w_0$$

Il risultato z viene di conseguenza passato ad una **funzione di attivazione** $g(z)$ per produrre l'output φ :

$$\varphi = g\left(\sum_{i=1}^n w_i x_i + w_0\right)$$

Osservazione 13. Nella sua forma originaria, la funzione di attivazione è la **funzione a scalino di Heaviside**:

$$g(x) = \begin{cases} 1, & \text{se } x \geq 0 \\ 0, & \text{se } x < 0 \end{cases}$$

e si può dimostrare che con ingressi binari $(0,1)$ è teoricamente sempre possibile trovare un opportuno insieme di pesi w tali da riprodurre le porte logiche fondamentali: AND, OR e NOT.

Poichè qualsiasi funzione logica può essere espressa combinando queste tre operazioni, connettendo più neuroni si può costruire qualsiasi circuito logico

Osservazione 14. Se i campioni del dataset sono linearmente separabili, esistono algoritmi di addestramento ad iterazioni **finite** che garantiscono di trovare i pesi corretti in un numero finito di passi.

Tuttavia il Perceptron è un classificatore strettamente lineare per cui, se le due classi non sono separabili da una retta o un iperpiano, l'algoritmo fallisce e non converge

Il limite teorico del Perceptron riguarda l'impossibilità di rappresentare la funzione logica XOR con un singolo neurone, infatti rappresentando tale funzione nel piano cartesiano con gli ingressi binari:

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
1	1	0
0	1	1
1	0	1

si nota che i punti appartenenti alla classe 1 e alla classe 0 si trovano su diagonali opposte e non possono essere separati da una retta.

Per superari dunque tali limiti si passa alle **reti neurali multistrato**, costituite dalla seguente architettura a strati:

1. **strato di ingresso:** rappresentante le n feature $x_1, \dots, x_n$ del campione;
2. **strati nascosti:** strati intermedi tra l'ingresso e l'uscita che elaborano l'informazione;
3. **strato d'uscita:** produce l'output finale stimato dalla rete

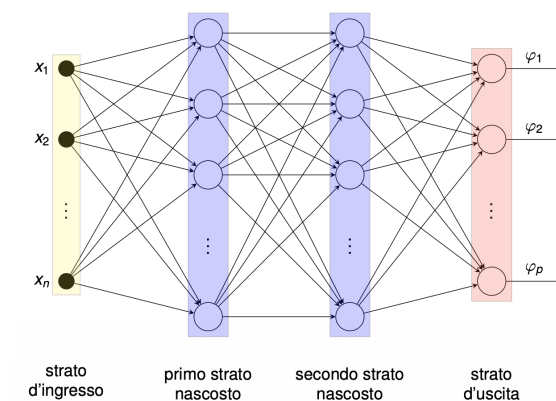


Figura 9.

Osservazione 15. In particolare si distingue tra reti **shallow**, caratterizzate da un solo strato nascosto, e reti **deep**, caratterizzate da molti strati nascosti

Enunciamo ora il seguente teorema:

Teorema 16. *Il teorema di approssimazione universale stabilisce che una rete neurale feedforward con un solo strato nascosto (shallow), contenente un numero finito ma sufficiente di neuroni e dotata di funzioni di attivazione non lineari, è in grado di approssimare qualsiasi funzione continua definita su un sottoinsieme compatto dello spazio $\mathbb{R}^n$ con un grado di precisione ε desiderato*

e sebbene esso sembri uno strumento molto potente, presenta forti limiti fisici e computazionali, in quanto per approssimare funzioni complesse con un solo strato, il numero di neuroni richiesti aumenterebbe esponenzialmente; aumentando invece la profondità della rete, essa è in grado di imparare in modo automatico a costruire una gerarchia di informazioni, passando da quelle più semplici a quelle più astratte.

Poichè per l'addestramento della rete occorre utilizzare metodi di ottimizzazione basati sul gradiente, la funzione scalino non è utilizzabile. Si introducono quindi funzioni di attivazione continue, limitate e derivabili:

- **funzione logistica:**

$$g(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

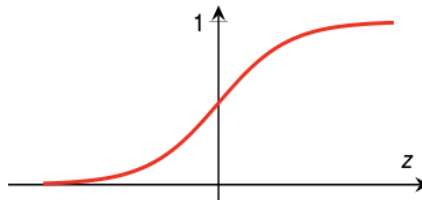


Figura 10.

- **tangente iperbolica:**

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$$

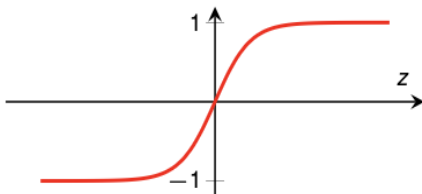


Figura 11.

Si fa però osservare come entrambe le funzioni soffrono del fenomeno di **Saturazione** , perchè per valori molto grandi o molto piccoli di z , la derivata della funzione tende a zero; ciò crea delle zone piatte nella funzione di costo, rendendo l'aggiornamento dei pesi basato sul gradiente inefficiente.

Si predilige allora la funzione **ReLU(Rectified Linear Unit)** che non satura per valori positivi ed è veloce da calcolare:

$$g(z) = \max \{0, z\}$$

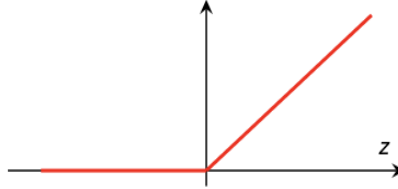


Figura 12.

1.5.1 Forward propagation

Presentiamo anzitutto la seguente simbologia per strati e neuroni:

- L : numero totale di strati nascosti;
- ℓ : indice dello strato per cui $\ell = 0$ indica lo strato di ingresso, $\ell = 1, \dots, L$ indica gli strati nascosti e $\ell = L + 1$ indica lo strato d'uscita;
- n_ℓ : numero di neuroni nello strato ℓ ;
- a_i^ℓ : uscita dell' i -esimo neurone dello strato ℓ ;
- w_{ij}^ℓ : peso dell'arco che collega il neurone j dello strato precedente $\ell - 1$ al neurone i dello strato corrente ℓ ;
- w_{i0}^ℓ : bias associato all' i -esimo neurone dello strato ℓ

Per ogni campione x , l'elaborazione procede in senso sequenziale dallo strato 0 allo strato $L + 1$:

- **strato di ingresso**: ogni neurone $i \in \{1, \dots, n\}$ produce un'uscita $a_i^0 = x_i$, in quanto si hanno direttamente le feature del campione, e la invia a ogni neurone j del primo strato nascosto attraverso un arco con peso w_{ij}^1 ;
- **strato nascosto**: ogni neurone $i \in \{1, \dots, n_\ell\}$ di uno strato nascosto $\ell \in \{1, \dots, L\}$ riceve l'uscita $a_j^{\ell-1}$ da ogni neurone j dello strato $\ell - 1$ attraverso un arco con peso w_{ij}^ℓ e calcola la somma pesata:

$$z_i^\ell = \sum_{j=1}^{n_\ell-1} w_{ij}^\ell a_j^{\ell-1} + w_{i0}^\ell$$

Di conseguenza produce un'uscita $a_i^\ell = g_i^\ell(z_i^\ell)$ usando una funzione di attivazione e la invia a ogni neurone j dello strato $\ell + 1$ attraverso un arco con peso $w_{ij}^{\ell+1}$;

- **strato di uscita:** ogni neurone $i \in \{1, \dots, p\}$ dello strato d'uscita riceve l'informazione a_j^L da ogni neurone j dell'ultimo strato nascosto L attraverso un arco con peso w_{ij}^{L+1} e calcola la somma pesata:

$$z_i^{L+1} = \sum_{j=1}^{n_L} w_{ij}^{L+1} a_j^L + w_{i0}^{L+1}$$

Di conseguenza produce un'uscita $a_i^{L+1} = g_i^{L+1}(z_i^{L+1})$ usando una funzione di attivazione in base al problema:

- per problemi di **regressione** si restituisce direttamente il valore lineare z_i^{L+1} ;
- per problemi di **classificazione** si applica una trasformazione d'uscita opportuna, come la softmax per la classificazione multiclasse:

$$g_i^{L+1}(z_i^{L+1}) = \frac{e^{z_i^{L+1}}}{\sum_{j=1}^p e^{z_j^{L+1}}}$$

o la sigmoide per quella binaria, in modo che i valori restituiti siano interpretabili come stime di probabilità

L'uscita della rete è dunque data da $\varphi_i = a_i^{L+1}, i = 1, \dots, p$.

Poichè addestrare una rete significa determinare il valore di tutti i pesi w e bias w_0 di ciascun arco, il numero totale di parametri del problema di ottimizzazione è:

$$\text{Parametri totali} = \sum_{\ell=1}^{L+1} n_{\ell}(n_{\ell-1} + 1)$$

A tal punto rappresentiamo l'architettura della rete neurale in modo compatto ed efficiente, per cui per ogni strato ℓ definiamo la matrice dei pesi $W^{\ell} \in \mathbb{R}^{n_{\ell} \times n_{\ell-1}}$ come segue:

$$W^{\ell} = \begin{pmatrix} (W^{\ell})_1 \\ \vdots \\ (W^{\ell})_{n_{\ell}} \end{pmatrix} = \begin{pmatrix} w_{11}^{\ell} & \dots & w_{1n_{\ell-1}}^{\ell} \\ \vdots & \ddots & \vdots \\ w_{n_{\ell}1}^{\ell} & \dots & w_{n_{\ell}n_{\ell-1}}^{\ell} \end{pmatrix}, \quad \ell = 1, \dots, L+1$$

e il vettore colonna di bias $b^{\ell} \in \mathbb{R}^{n_{\ell} \times 1}$:

$$b^{\ell} = (w_{10}^{\ell} \dots w_{n_{\ell}0}^{\ell})^{\top}, \quad \ell = 1, \dots, L+1$$

Si hanno anche il vettore delle combinazioni lineari $z^{\ell} \in \mathbb{R}^{n_{\ell} \times 1}$ che raccoglie la somma pesata più il bias per ciascun neurone dello strato ℓ :

$$z^{\ell} = (z_1^{\ell} \dots z_{n_{\ell}}^{\ell})^{\top}, \quad \ell = 1, \dots, L+1$$

e il vettore delle attivazioni $a^{\ell} \in \mathbb{R}^{n_{\ell} \times 1}$, contenente i valori di output inviati dai neuroni dello strato ℓ allo strato successivo:

$$a^{\ell} = (a_1^{\ell} \dots a_{n_{\ell}}^{\ell})^{\top}, \quad \ell = 0, \dots, L+1$$

ottenuto applicando la funzione di attivazione $g^{\ell}(z^{\ell}) \in \mathbb{R}^{n_{\ell} \times 1}$:

$$g^{\ell}(z^{\ell}) = (g_1^{\ell}(z_1^{\ell}) \dots g_{n_{\ell}}^{\ell}(z_{n_{\ell}}^{\ell}))^{\top}, \quad \ell = 1, \dots, L+1$$

Il calcolo dell'uscita si riduce dunque all'algoritmo di **forward propagation**, il quale consiste nel far scorrere l'informazione dallo strato di ingresso fino allo strato di uscita attraverso un ciclo iterativo sugli strati:

Algoritmo 1

```

Poni  $a^0 = x$ 
For  $\ell = 1, \dots, L$ 
    Calcola  $z^\ell = W^\ell a^{\ell-1} + b^\ell$ 
    Calcola  $a^\ell = g^\ell(z^\ell)$ 
End for
Calcola  $z^{L+1} = W^{L+1} a^L + b^{L+1}$ 
Calcola  $\varphi$  da  $z^{L+1}$ 

```

1.5.2 Back propagation

Una volta calcolata l'uscita della rete neurale, è necessario addestrarla minimizzando il rischio empirico mediante metodi di ottimizzazione del primo ordine, per cui definiamo una diversa loss function a seconda del problema; per ogni campione x , misuriamo l'errore E tra l'output φ e l'etichetta y come:

$$E = \begin{cases} \frac{1}{2} \|\varphi - y\|^2, & \text{se il problema è di regressione (errore quadratico)} \\ -\sum_{t=1}^p y_t \log(\varphi_t), & \text{se il problema è di classificazione (cross entropy)} \end{cases}$$

e ricordiamo che per definizione di rischio empirico

$$R(w) = \frac{1}{m} \sum_{i=1}^m E_i(w)$$

segue:

$$\nabla R(w) = \frac{1}{m} \sum_{i=1}^m \nabla E_i(w)$$

ove possiamo calcolare $\nabla E_i(w)$ attraverso una procedura di back propagation.

Innanzitutto notiamo che l'errore $E_i(w_{ij}^\ell)$ dipende dal peso w_{ij}^ℓ attraverso z_i^ℓ , per cui usando la regola della catena possiamo scrivere:

$$\frac{\partial E}{\partial w_{ij}^\ell} = \frac{\partial E}{\partial z_i^\ell} \frac{\partial z_i^\ell}{\partial w_{ij}^\ell}$$

così da dividere il problema in due fattori distinti.

Teorema 17. *Per ogni strato $\ell \in \{1, \dots, L+1\}$ e per ogni coppia di neuroni (i, j) , con $i \in \{1, \dots, n_\ell\}$ e $j \in \{1, \dots, n_{\ell+1}\}$, si ha che:*

$$\frac{\partial E}{\partial w_{ij}^\ell} = \frac{\partial E}{\partial z_i^\ell} \frac{\partial z_i^\ell}{\partial w_{ij}^\ell}$$

ove:

$$\frac{\partial E}{\partial z_i^\ell} = \begin{cases} \varphi_i - y_i, & \text{se } \ell = L+1 \\ \frac{dg_i^\ell(z_i^\ell)}{dz_i^\ell} \sum_{h=1}^{n_{\ell+1}} w_{hi}^{\ell+1} \frac{\partial E}{\partial z_h^{\ell+1}}, & \text{se } \ell \leq L \end{cases}$$

$$\frac{\partial z_i^\ell}{\partial w_{ij}^\ell} = \begin{cases} 1, & \text{se } j = 0 \\ a_j^{\ell-1}, & \text{se } j \geq 1 \end{cases}$$

Dimostrazione. Poichè la combinazione lineare per il neurone i è:

$$z_i^\ell = \sum_{h=1}^{n_\ell-1} w_{ih}^\ell a_h^{\ell-1} + w_{i0}^\ell$$

derivando rispetto a w_{ij}^ℓ si ottiene semplicemente:

$$\frac{\partial z_i^\ell}{\partial w_{ij}^\ell} = \begin{cases} 1, & \text{se } j=0 \\ a_j^{\ell-1}, & \text{se } j \geq 1 \end{cases}$$

ove per $j \geq 1$ il valore dell'output del neurone j dello strato precedente $\ell - 1$ è già stato calcolato durante la fase di forward propagation.

Per quanto riguarda invece l'errore commesso relativo al potenziale z_i^ℓ , $\frac{\partial E}{\partial z_i^\ell}$, lo si può calcolare distinguendo tra strato di uscita e strati nascosti:

- se $\ell = L + 1$, dall'espressione della loss function e dell'uscita $\varphi_i = a_i^{L+1}$, $i = 1, \dots, p$, si può mostrare che la derivata rispetto al potenziale di uscita z_i^{L+1} fornisca la stessa identica espressione:

$$\frac{\partial E}{\partial z_i^{L+1}} = \varphi_i - y_i = a_i^{L+1} - y_i$$

ovvero la differenza tra la predizione dell'uscita e il valore del target vero, sia nel caso della regressione che in quello della classificazione multiclasse;

- se $\ell \leq L$, notiamo che un cambiamento in z^ℓ influenza le uscite di tutti i neuroni dello strato successivo $\ell + 1$, per cui l'errore E dipende da z^ℓ attraverso $z^{\ell+1}$ e usando la regola della catena segue:

$$\frac{\partial E}{\partial z^\ell} = \frac{\partial z^{\ell+1}}{\partial z^\ell} \frac{\partial E}{\partial z^{\ell+1}}$$

ove il termine $\frac{\partial E}{\partial z^{\ell+1}}$ relativo allo strato successivo è già stato calcolato nel passo precedente dell'algoritmo, procedendo a ritroso. L'informazione d'errore viene quindi propagata all'indietro dallo strato di uscita verso i primi strati della rete.

Poichè invece $z^{\ell+1} = W^{\ell+1} a^\ell + b^{\ell+1} = W^{\ell+1} g(z^\ell) + b^{\ell+1}$ segue:

$$\frac{\partial z^{\ell+1}}{\partial z^\ell} = \frac{d g^\ell(z^\ell)}{d z^\ell} (W^{\ell+1})^\top$$

da cui:

$$\frac{\partial E}{\partial z^\ell} = \frac{d g^\ell(z^\ell)}{d z^\ell} (W^{\ell+1})^\top \frac{\partial E}{\partial z^{\ell+1}} = \begin{pmatrix} \frac{d g_1^\ell(z_1^\ell)}{d z_1^\ell} (w_{11}^{\ell+1} \dots w_{n_{\ell+1}1}^{\ell+1}) \\ \vdots \\ \frac{d g_{n_\ell}^\ell(z_{n_\ell}^\ell)}{d z_{n_\ell}^\ell} (w_{1n_\ell}^{\ell+1} \dots w_{n_{\ell+1}n_\ell}^{\ell+1}) \end{pmatrix} \frac{\partial E}{\partial z^{\ell+1}}$$

Espanendo dunque il calcolo all' i -esimo neurone si ha che:

$$\frac{\partial E}{\partial z_i^\ell} = \frac{d g_i^\ell(z_i^\ell)}{d z_i^\ell} \sum_{h=1}^{n_{\ell+1}} w_{hi}^{\ell+1} \frac{\partial E}{\partial z_h^{\ell+1}}$$

□

Vediamo infine la struttura completa dell'algoritmo di back propagation:

Algoritmo 2

```

Calcola  $\varphi$  da forward propagation
For  $\ell = L + 1, \dots, 1$ 
  For  $i = 1, \dots, n_\ell$ 
    Calcola  $\frac{\partial E}{\partial z_i^\ell}$ 
    For  $j = 0, \dots, n_{\ell-1}$ 
      Calcola  $\frac{\partial z_i^\ell}{\partial w_{ij}^\ell}$ 
      Poni  $\frac{\partial E}{\partial w_{ij}^\ell} = \frac{\partial E}{\partial z_i^\ell} \frac{\partial z_i^\ell}{\partial w_{ij}^\ell}$ 
    End for
  End for
End for
End for

```

Nello specifico tale algoritmo come primo passo calcola e memorizza tutti i vettori z^ℓ e tutte le attivazioni a^ℓ per ciascuno strato, a partire dall'input $a^0 = x$; dopodichè, per $\ell = L + 1, \dots, 1$, esegue un primo ciclo sui neuroni dello strato attuale calcolando la sensibilità del potenziale $\frac{\partial E}{\partial z_i^\ell}$ e verificando se lo strato è di uscita o meno. Infine esegue un ciclo sui neuroni dello strato precedente e calcola la derivata rispetto al peso.

1.6 Metodi di ottimizzazione per il machine learning

Consideriamo il problema di minimizzazione:

$$\min_w \{R(w) + \lambda \Omega(w)\}$$

ove $R(w) = \frac{1}{m} \sum_{i=1}^m E_i(w)$ e solitamente $\Omega(w) = \|w\|^2$; in questa sezione però non tratteremo il termine di regolarizzazione per cui ci concentreremo sul problema $\min_w \{R(w)\}$.

Introduciamo innanzitutto la distinzione fondamentale tra due grandi approcci di ottimizzazione:

- **metodi batch:** a ogni iterazione dell'algoritmo si calcola il gradiente esatto della funzione di rischio empirico $\nabla R(w)$ utilizzando tutti gli m campioni del dataset.

Essi sono adatti a situazioni in cui la dimensione del problema e del modello è contenuta e il tempo per iterazione è ragionevole rispetto al budget temporale dato.

Nonostante ciò potrebbe rappresentare uno svantaggio se le dimensioni del dataset sono enormi, tali metodi permettono di applicare le tecniche note di ottimizzazione non lineare, hanno buona rapidità di convergenza e poichè il gradiente totale è la media dei gradienti sui singoli campioni, il calcolo può essere distribuito su più GPU;

- **metodi stocastici:** invece di usare tutti gli m campioni, a ogni iterazione si seleziona casualmente un sottoinsieme ridotto, chiamato **mini batch**, e su di esso si calcola una stima del gradiente vero.

In tal modo il costo computazionale della singola iterazione diminuisce ma si introduce una minore precisione nella direzione di discesa

Chiariamo anche la nozione formale e pratica di **epoca**: l'epoca è il numero di iterazioni necessarie affinché l'algoritmo stocastico veda un numero complessivo di campioni pari alla dimensione dell'intero training set; ad esempio se abbiamo $m = 1000$ campioni e usiamo un mini batch di dimensione 10, un'epoca corrisponde a 100 iterazioni stocastiche.

Osservazione 18. Poichè la scelta dei mini batch avviene in maniera casuale non è garantito che dopo 100 iterazioni si vedano tutti i campioni m indistintamente

Siccome in un contesto di Big Data il calcolo del gradiente $\nabla R(w)$ è oneroso, si è deciso di utilizzare il **metodo del gradiente stocastico**: esso calcola una stima $\tilde{\nabla} R(w)$ e la utilizza come direzione di discesa per la line search, $w_{k+1} = w_k - \alpha_k \tilde{\nabla} R(w_k)$.

Ricordando poi la definizione di rischio empirico possiamo porre:

$$\tilde{\nabla} R(w_k) = \frac{1}{|S_k|} \sum_{i \in S_k} \nabla E_i(w_k)$$

con $S_k \subseteq \{1, \dots, m\}$ sottoinsieme di campioni.

Si può mostrare i vantaggi sono dati da:

1. il costo computazionale è inferiore in quanto si utilizza un sottoinsieme S_k di campioni;
2. spesso le informazioni contenute nei campioni sono ridondanti

mentre uno svantaggio è dato dal fatto che la funzione obiettivo inizialmente decresce in modo rapido, ma quando ci si avvicina al minimo le oscillazioni aumentano a causa della varianza intrinseca della stima stocastica.

In particolare si hanno quattro assunzioni fondamentali per l'analisi di convergenza del metodo del gradiente stocastico:

- si assume che la funzione obiettivo $R(w)$ sia continuamente differenziabile e che il suo gradiente $\nabla R(w)$ sia Lipschitz continuo con costante L . Ciò garantisce che la derivata, ovvero la pendenza, non cambi in maniera brusca o discontinua;
- esistono due costanti $0 < c_1 \leq c_2$ tali che, per ogni $k \geq 0$, le condizioni d'angolo estese al caso stocastico siano rispettate:

$$\begin{cases} \nabla R(w_k)^\top \mathbb{E}[\tilde{\nabla} R(w_k)|w_k] \geq c_1 \|\nabla R(w_k)\|^2 \\ \|\mathbb{E}[\tilde{\nabla} R(w_k)|w_k]\| \leq c_2 \|\nabla R(w_k)\| \end{cases}$$

- la varianza della stima stocastica è limitata superiormente e generalmente si assume l'esistenza di due costanti $M_1 \geq 0, M_2 \geq 1$ tali che:

$$\mathbb{V}[\tilde{\nabla} R(w_k)|w_k] \leq M_1 + M_2 \|\nabla R(w_k)\|^2$$

ove il parametro M_1 rappresenta l'entità del rumore stocastico dovuto al fatto che non si usano tutti i campioni.

In base a queste assunzioni nel caso non convesso, le proprietà di convergenza dipendono dalla scelta della lunghezza del passo α_k :

- con un passo costante $\alpha_k = \alpha$, in assenza di rumore si ha convergenza al gradiente nullo, altrimenti l'algoritmo non raggiunge un punto stazionario esatto, ma oscilla all'interno di una regione di tolleranza attorno al minimo;
- con un passo decrescente $\alpha_k \propto \frac{1}{k}$ e rispettando le condizioni di convergenza stocastica:

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \quad \sum_{k=0}^{\infty} \alpha_k^2 < \infty$$

si annulla l'effetto del rumore asintoticamente, garantendo che:

$$\lim_{k \rightarrow \infty} \inf \mathbb{E}[\|\nabla R(w_k)\|^2] = 0$$

ma la convergenza diventa sub-lineare

Per superare i limiti di tale metodo, come oscillazioni e lentezza nelle zone piatte, sono state proposte differenti varianti:

- **momentum**: permette di accumulare le direzioni precedenti per superare zone piatte e smorzare le oscillazioni ad alta frequenza, definito dall'iterazione:

$$w_{k+1} = w_k - \alpha_k \tilde{\nabla} R(w_k) + \beta(w_k - w_{k-1}), \quad \beta > 0$$

- **adam (adaptive moment estimation)**: combina l'idea del momentum con quella di stima della varianza dei gradienti per adattare il learning rate componente per componente;
- **gradient aggregation**: ideati per ridurre la varianza mantenendo la stima non distorta, tra cui:
 - **SVRG (Stochastic Variance Reduced Gradient)**: ogni K iterazioni, l'algoritmo calcola e memorizza il gradiente completo in un punto fisso $\tilde{w}$. Nelle iterazioni intermedie, corregge la stima stocastica sottraendo il gradiente del singolo campione nel punto fisso e aggiungendo il gradiente completo memorizzato;
 - **SAGA**: invece di calcolare il gradiente completo ogni volta, mantiene in memoria un'approssimazione aggiornata dei gradienti di tutti i campioni estratti in precedenza

Osservazione 19. Per la scelta del passo α_k , nel metodo del gradiente stocastico è anche possibile usare il passo di Poliak o metodi stocastici del secondo ordine, ad esempio Newton troncato

1.7 Reti neurali per il deep learning generativo

A differenza dei modelli tradizionali, che imparano a classificare i dati, i **modelli generativi** hanno lo scopo di produrre nuovi dati che siano coerenti e simili a quelli presenti nel dataset di partenza. Un particolare approccio si basa sull'architettura delle **GAN (Generative Adversarial Nets)**; questa tecnica sfrutta la competizione tra due reti neurali distinte:

- **il discriminatore** D : classificatore binario, $D: \mathcal{X} \rightarrow [0, 1]$, che prende in ingresso un campione x e restituisce in output la probabilità che x sia un dato reale oppure un dato falso prodotto dal generatore;
- **il generatore** G : funzione $G: \mathcal{Z} \rightarrow \mathcal{X}$ che prende in ingresso un vettore $z \in \mathcal{Z}$ estratto casualmente da una distribuzione a proprio sul rumore, $p_z(z)$, e lo trasforma in un campione nello spazio dei dati $\mathcal{X}$

La funzione di costo e l'addestramento congiunto delle due reti vengono modellati come un problema di ottimizzazione **minimax**:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log(D(x))] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

e analizzando in dettaglio i singoli componenti dell'equazione si osserva che:

- il discriminatore D vuole massimizzare la funzione per cui cerca di assegnare 1 ai dati reali, $D(x) \rightarrow 1$, e 0 ai dati generati, $D(G(z)) \rightarrow 0$;
- il generatore G vuole minimizzare la funzione per cui vuole che il termine $D(G(z))$ sia il più possibile vicino a 1 così che il logaritmo assume un valore negativo molto grande, minimizzando l'espressione

All'ottimo però D non è più in grado di distinguere se x è un dato reale; allora possiamo definire un metodo tipo gradiente stocastico alternando la minimizzazione con la massimizzazione:

Algoritmo 3

```
For  $k = 0, 1, \dots$ 
  Scegli  $r$  campioni  $\{z^{(1)}, \dots, z^{(r)}\}$  da  $p_z(z)$ 
  Scegli  $r$  campioni  $\{x^{(1)}, \dots, x^{(r)}\}$  da  $p_{\text{data}}(x)$ 
  Calcola una stima del gradiente per  $D$ 
  Aggiorna i parametri di  $D$  con un'iterazione del gradiente stocastico
  Scegli  $r$  campioni  $\{z^{(1)}, \dots, z^{(r)}\}$  da  $p_z(z)$ 
  Calcola una stima del gradiente per  $G$ 
  Aggiorna i parametri di  $G$  con un'iterazione del gradiente stocastico
End for
```

1.8 Cenni sull'apprendimento non supervisionato

Nei problemi di apprendimento non supervisionato non si dispone delle etichette y^i per cui non si hanno informazioni sulla classe di appartenenza dei campioni e di conseguenza l'algoritmo deve scoprire autonomamente strutture o raggruppamenti intrinseci nei dati basandosi esclusivamente sulle caratteristiche dei campioni stessi e su misure di **similarità** o **dissimilarità**.

A tal proposito viene utilizzata la tecnica **cluster analysis**: essa prevede di suddividere un insieme di m campioni in k gruppi, detti cluster, in modo che:

- campioni appartenenti allo stesso cluster siano il più simili possibile tra loro;
- campioni appartenenti a cluster diversi siano il più dissimili possibile

Osservazione 20. *La cluster analysis appartiene alla classe di problemi computazionalmente difficili, detti NP-hard*

Osservazione 21. *Non esiste una definizione unica di cosa costituisca un cluster, motivo per cui il metodo che utilizzeremo sarà quello basato sui **centroidi***

Nello specifico si distingue tra:

- **hard clustering**: ogni punto appartiene esattamente a un cluster;
- **soft clustering**: ogni punto può appartenere a più cluster

e per semplicità ci concentreremo sui cluster di primo tipo; si vuole dunque trovare una partizione dell'insieme dei campioni $\mathcal{T} = \{x^1, \dots, x^m\}$ in k cluster $C_1, \dots, C_k$ tali che:

$$\begin{cases} C_i \subseteq \mathcal{T} \neq \emptyset, & i = 1, \dots, k \\ \bigcup_{i=1}^k C_i = \mathcal{T} \\ C_i \cap C_j = \emptyset, & i \neq j \end{cases}$$

Nella pratica applicativa il **K-means** è uno dei metodi basati su centroidi più diffuso, dovuto alla sua estrema semplicità ed efficienza; l'algoritmo assegna m campioni a k cluster definendo k punti nello spazio chiamati **centroidi**, $z^1, \dots, z^k \in \mathbb{R}^n$, e delle variabili binarie di appartenenza $w_{ij} \in \{0, 1\}$, $i = 1, \dots, m, j = 1, \dots, k$:

- $w_{ij} = 1$ se il campione x^i appartiene al cluster j , altrimenti $w_{ij} = 0$;
- ogni punto deve appartenere a uno e un solo cluster:

$$\sum_{j=1}^k w_{ij} = 1, \quad \forall i$$

Per determinare allora la funzione obiettivo si fa osservare come la bontà della partizione dipende da quanto i punti sono vicini al proprio centroide, per cui una buona partizione minimizza la somma delle distanze tra i punti e il centroide del cluster a cui appartengono.

Ovviamente per minimizzare la distanza di un punto x^i rispetto al suo centroide più vicino il problema è dato da:

$$\min_{j=1, \dots, k} \{d(x^i, z^j)\}$$

ma poichè, fissato il centroide, esso è buono se tutti i punti x^i che gli appartengono sono a lui vicino, il problema da risolvere si riduce a:

$$\min_{z^1, \dots, z^k \in \mathbb{R}^n} \left(\sum_{i=1}^m \min_{j=1, \dots, k} \{d(x^i, z^j)\} \right)$$

o equivalentemente, utilizzando le variabili w_{ij} , si ha che:

$$\min_z \sum_{i=1}^m \left(\min_w \sum_{j=1}^k w_{ij} d(x^i, z^j) \right)$$

Il problema di ottimizzazione k-means minimizza però la somma delle distanze euclidee al quadrato tra ciascun punto e il centroide del cluster a cui è assegnato:

$$\min_z \sum_{i=1}^m \left(\min_w \sum_{j=1}^k w_{ij} \|x^i - z^j\|^2 \right)$$

Tuttavia la funzione obiettivo coinvolge simultaneamente variabili continue z e variabili discrete w , per cui il problema globale è non convesso. Viene risolto in maniera iterativa alternando due passi in forma chiusa:

1. **aggiornamento dei centroidi a pesi fissati:** si mantengono fisse le assegnazioni dei punti ai cluster e rispetto ai centroidi z^j il problema diventa un problema di minimizzazione quadratico e convesso non vincolato.

Annullando il gradiente rispetto a ciascun centroide, si ottiene che la posizione ottimale del centroide è semplicemente la **media aritmetica** di tutti i punti appartenenti a quel cluster C_j :

$$z^j = \frac{1}{|C_j|} \sum_{x^i \in C_j} x^i, \quad j = 1, \dots, k$$

2. **aggiornamento dei pesi a centri fissati:** si mantengono fisse le posizioni dei centroidi e ogni punto x^i viene riassegnato al centroide z^j più vicino in termini di distanza euclidea:

$$w_{ij} = \begin{cases} 1, & \text{se } j = \bar{j} \\ 0, & \text{altrimenti} \end{cases}$$

ove $\bar{j} \in \text{Arg min}_{j=1, \dots, k} \|x^i - z^j\|$, per $i = 1, \dots, m$

Osservazione 22. Ad ogni iterazione, sia aggiornando z che aggiornando w , il valore della funzione obiettivo decresce in modo monotono.

Poiché il numero di modi possibili di assegnare m campioni a k cluster è finito, l'algoritmo termina necessariamente in un numero finito di passi senza rischiare di entrare in cicli infiniti.

Inoltre la soluzione (z^, w^*) restituita non è necessariamente un minimo globale, ma un **ottimo parziale**, perchè per individuare il minimo globale si utilizzano tipicamente più ripartenze con posizioni iniziali casuali*